

Министерство науки и высшего образования
Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №7
«Методы оптимизации в нейросетевых вычислениях»
по дисциплине
«Методы оптимизации»
Вариант №5

Проверил:
Кудашов Вячеслав Николаевич

Выполнила:
Косогова Мария Александровна
Группа: Р3206
Поток: 1.1

Санкт-Петербург, 2024

Содержание

1	Задание 1. Аналитическое исследование функции	2
1.1	Нахождение стационарных точек	2
1.2	Исследование типа стационарной точки	2
1.3	Нахождение глобального минимума	3
1.4	Линии уровня функции	3
1.5	Вывод по заданию 1	4
2	Задание 2. Градиентный спуск и Adagrad	4
2.1	Обычный градиентный спуск	4
2.2	Собственная реализация Adagrad	5
2.3	Подбор гиперпараметров	5
2.4	Готовая реализация Adagrad из PyTorch	6
2.5	Вывод по заданию 2	7
3	Задание 3. Обучение нейронной сети	7
3.1	Описание задачи и датасета	7
3.2	Архитектура нейронной сети	7
3.3	Функция потерь и метрика качества	8
3.4	Сравниваемые оптимизаторы	8
3.5	Результаты обучения	8
3.6	Вывод по заданию 3	8

1 Задание 1. Аналитическое исследование функции

Дана функция по варианту 5:

$$f(x, y) = 10^{-2}(8x^2 + 2xy - 23x - 6y - 3)$$

Область определения с учётом ограничения:

$$(x, y) \in [-20, 20] \times [-50, 50]$$

1.1 Нахождение стационарных точек

Найдём частные производные первого порядка:

$$f_x = 10^{-2}(16x + 2y - 23)$$

$$f_y = 10^{-2}(2x - 6)$$

Стационарные точки определяются из системы:

$$\begin{cases} 16x + 2y - 23 = 0 \\ 2x - 6 = 0 \end{cases}$$

Из второго уравнения получаем:

$$2x = 6 \implies x = 3$$

Подставим найденное значение в первое уравнение:

$$16(3) + 2y - 23 = 0$$

$$48 + 2y - 23 = 0$$

$$2y + 25 = 0 \implies y = -12,5$$

Следовательно, функция имеет единственную стационарную точку:

$$(3; -12,5)$$

1.2 Исследование типа стационарной точки

Найдём матрицу Гессе:

$$H = 10^{-2} \begin{pmatrix} 16 & 2 \\ 2 & 0 \end{pmatrix} = \begin{pmatrix} 0,16 & 0,02 \\ 0,02 & 0 \end{pmatrix}$$

Определитель матрицы Гессе:

$$D = 0,16 \cdot 0 - 0,02^2 = -0,0004$$

Так как определитель матрицы Гессе отрицательный, матрица Гессе является неопределённой. Следовательно, стационарная точка

$$(3; -12,5)$$

является седловой точкой. Локальных минимумов и локальных максимумов нет.

1.3 Нахождение глобального минимума

Функцию можно представить в виде:

$$f(x, y) = 10^{-2}(8x^2 - 23x - 3 + y(2x - 6))$$

Для фиксированного значения x функция является линейной по y , поэтому минимум по y достигается на границе области.

Если $2x - 6 > 0$ (то есть $x > 3$), то минимум достигается при:

$$y = -50$$

Если $2x - 6 < 0$ (то есть $x < 3$), то минимум достигается при:

$$y = 50$$

Если $x = 3$, то функция не зависит от y .

Рассмотрим случай $y = 50$, где $x \in [-20, 3]$:

$$f(x, 50) = 10^{-2}(8x^2 - 23x - 3 + 50(2x - 6)) = 10^{-2}(8x^2 + 77x - 303)$$

Вершина параболы:

$$x = -\frac{77}{16} = -4,8125$$

Значение функции в этой точке:

$$f(-4,8125; 50) = 10^{-2}(8(-4,8125)^2 + 77(-4,8125) - 303) = -4,8828125$$

Рассмотрим случай $y = -50$, где $x \in [3, 20]$:

$$f(x, -50) = 10^{-2}(8x^2 - 23x - 3 - 50(2x - 6)) = 10^{-2}(8x^2 - 123x + 297)$$

Вершина параболы:

$$x = \frac{123}{16} = 7,6875$$

Значение функции в этой точке:

$$f(7,6875; -50) = 10^{-2}(8(7,6875)^2 - 123(7,6875) + 297) = -1,7578125$$

Сравнивая минимумы на границах ($-4,8828125 < -1,7578125$), глобальный минимум функции на заданной области достигается в точке:

$$(x_{min}, y_{min}) = (-4,8125; 50)$$

Значение глобального минимума:

$$f_{min} = -4,8828125$$

1.4 Линии уровня функции

На рисунке 1 представлены линии уровня исследуемой функции.

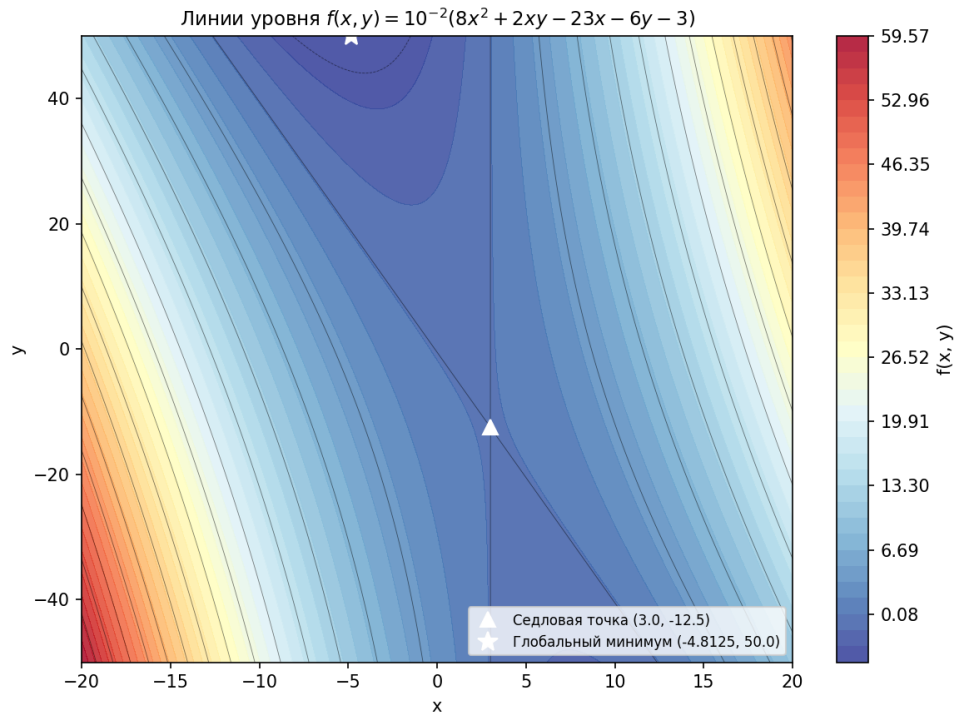


Рис. 1: Линии уровня функции

1.5 Вывод по заданию 1

Функция имеет одну стационарную точку:

$$(3; -12, 5)$$

Эта точка является седловой. Локальных экстремумов нет. Глобальный минимум достигается на границе заданной области:

$$(-4, 8125; 50)$$

Такая функция может представлять сложность для обычного градиентного спуска, так как метод может сходиться к седловой точке и останавливаться в её окрестности.

2 Задание 2. Градиентный спуск и Adagrad

2.1 Обычный градиентный спуск

Для обычного градиентного спуска используется формула:

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

В данной работе были выбраны следующие параметры, чтобы алгоритм застревал в седловой точке за ~ 100 итераций:

$$(x_0, y_0) = (10; -12)$$

$$\alpha = 0,5$$

$$N = 100$$

Седловая точка имеет координаты $(3; -12, 5)$. Начальное приближение и скорость обучения были выбраны так, чтобы обычный градиентный спуск пришёл в окрестность седловой точки.

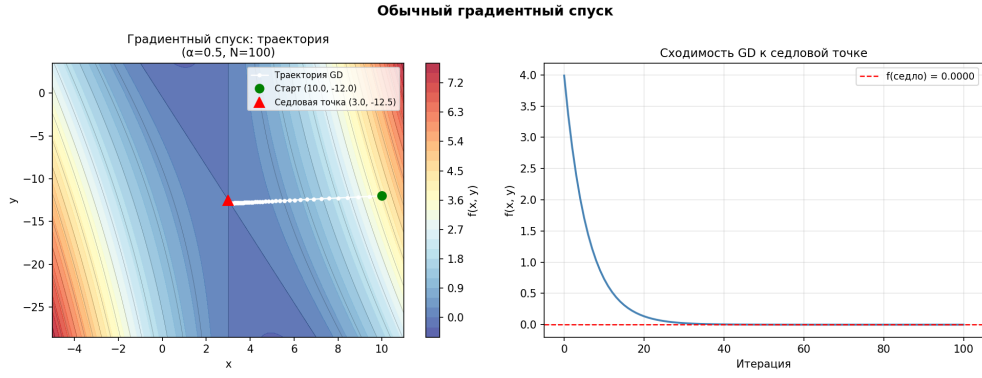


Рис. 2: Сходимость обычного градиентного спуска к седловой точке

2.2 Собственная реализация Adagrad

Метод Adagrad (Adaptive Gradient) адаптирует скорость обучения для каждого параметра, накапливая сумму квадратов исторических градиентов. Формулы метода:

$$G_t = G_{t-1} + g_t \odot g_t$$

Обновление параметров:

$$x_{t+1} = x_t - \frac{\alpha}{\sqrt{G_t + \varepsilon}} \odot g_t$$

Где g_t — градиент на текущем шаге, G_t — диагональная матрица (вектор) накопленных квадратов градиентов, ε — сглаживающий параметр для избежания деления на ноль. В работе использовались параметры:

$$\alpha = 5, 0$$

$$\varepsilon = 10^{-8}$$

На рисунке 3 показана траектория метода Adagrad. Видно, что последовательность приближений преодолевает окрестность седловой точки и продолжает движение к глобальному минимуму.

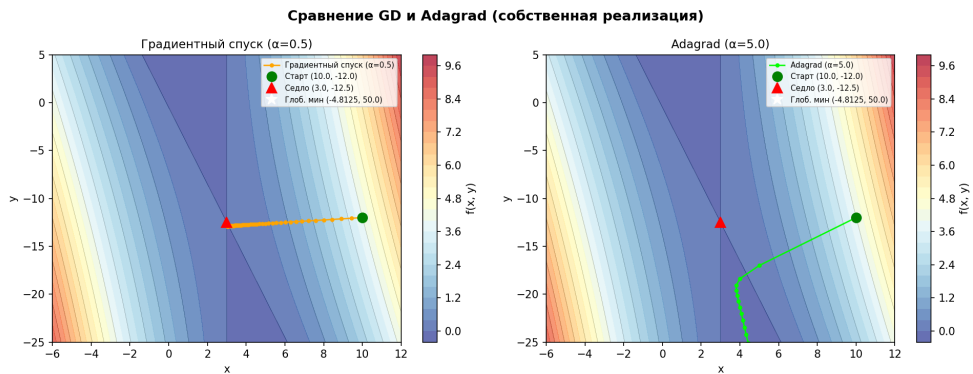


Рис. 3: Собственная реализация Adagrad

2.3 Подбор гиперпараметров

Для более наглядного отображения траектории была выбрана локальная область вблизи седловой точки. Количество итераций в обоих случаях равно $N = 100$.

Для получения **пилообразной траектории** (движение в стороны от направления) была выбрана ббольшая скорость обучения:

$$\alpha = 15,0$$

Для получения **более плавной траектории** без значительных колебаний скорость обучения была уменьшена:

$$\alpha = 3,0$$

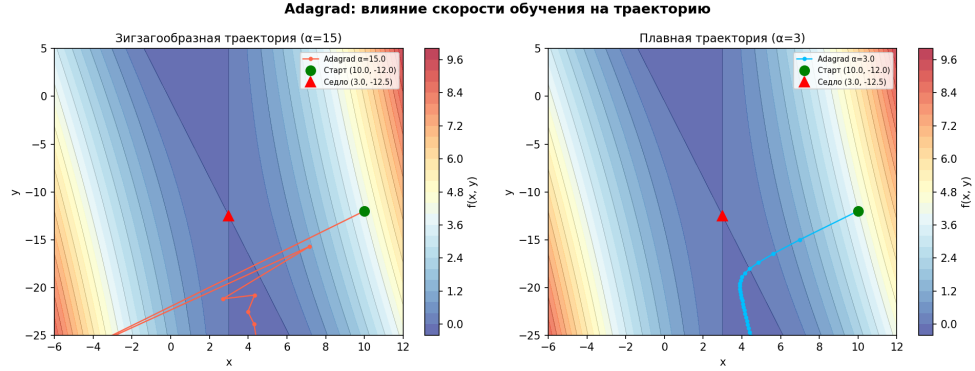


Рис. 4: Adagrad: пилообразная ($\alpha=15$) и плавная ($\alpha=3$) траектории

2.4 Готовая реализация Adagrad из PyTorch

Для проверки была использована готовая реализация оптимизатора Adagrad из библиотеки PyTorch: `torch.optim.Adagrad`

Параметры:

$$\text{lr} = 5,0$$

$$\text{eps} = 10^{-8}$$

Результат работы готовой реализации успешно преодолевает окрестность седловой точки, демонстрируя поведение, аналогичное собственной реализации.

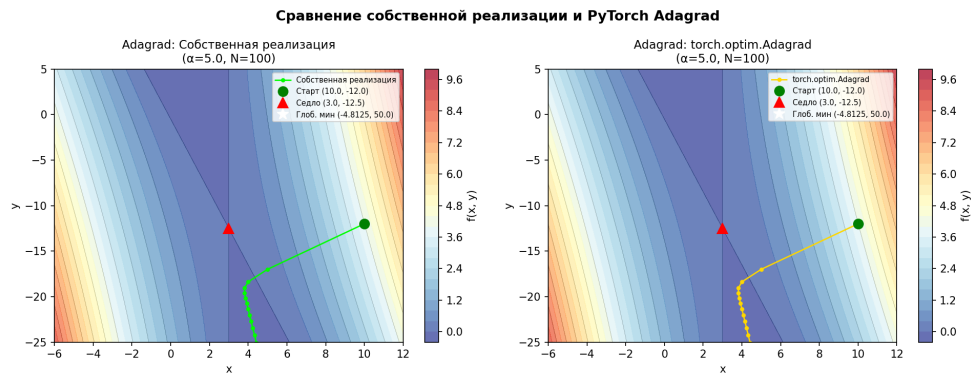


Рис. 5: Готовая реализация PyTorch Adagrad

2.5 Вывод по заданию 2

Обычный градиентный спуск при выбранных параметрах сходится к седловой точке, что наглядно иллюстрирует проблему застревания оптимизационных методов в невыпуклых задачах.

Метод Adagrad, использующий адаптивную скорость обучения на основе накопления градиентов, позволяет преодолеть влияние седловой точки и продолжить спуск в сторону глобального минимума. Изменяя базовую скорость обучения (learning rate), можно регулировать характер траектории от гладкой до пилообразной. Готовая реализация из PyTorch ведёт себя консистентно с написанным вручную алгоритмом.

3 Задание 3. Обучение нейронной сети

3.1 Описание задачи и датасета

В задании 3 была выбрана задача многоклассовой классификации изображений на датасете **Intel Image Classification** (источник: Kaggle, дата публикации: 2018 год). Датасет содержит около 25 000 цветных изображений (RGB, 3 канала) размером 150×150 пикселей. Данные распределены по 6 природным и урбанистическим классам:

1. Buildings (Здания)
2. Forest (Лес)
3. Glacier (Ледник)
4. Mountain (Горы)
5. Sea (Море)
6. Street (Улица)

Для ускорения обучения в рамках лабораторной работы изображения были масштабированы до размера 64×64 пикселя.

3.2 Архитектура нейронной сети

Для извлечения пространственных признаков из цветных изображений была построена сверточная нейронная сеть (CNN). Она состоит из следующих слоёв:

- сверточный слой Conv2d (вход: 3 канала, выход: 16 фильтров, ядро 3×3);
- функция активации ReLU;
- слой подвыборки MaxPool2d (ядро 2×2);
- сверточный слой Conv2d (вход: 16 каналов, выход: 32 фильтра, ядро 3×3);
- функция активации ReLU;
- слой подвыборки MaxPool2d (ядро 2×2);
- слой сглаживания (Flatten);
- полносвязный слой Linear (выход: 128 нейронов);

- функция активации ReLU;
- слой регуляризации Dropout ($p = 0.4$) для предотвращения переобучения;
- выходной полносвязный слой Linear на 6 классов.

3.3 Функция потерь и метрика качества

Так как решается задача многоклассовой классификации, в качестве функции потерь была использована перекрестная энтропия:

$$Loss = \text{CrossEntropyLoss}$$

В качестве метрики качества использовалась доля правильных ответов (Accuracy):

$$Accuracy = \frac{\text{число правильных ответов}}{\text{общее число объектов}}$$

3.4 Сравнимые оптимизаторы

В работе сравнивались два оптимизатора:

1. готовая реализация `torch.optim.Adagrad`
2. собственная реализация `MyAdagrad`, наследующаяся от базового класса `torch.optim.Optimizer`

Оба оптимизатора использовали одинаковую начальную скорость обучения $\text{lr} = 0,005$. Размер батча (batch size) составил 64. Обучение проводилось в течение 10 эпох.

3.5 Результаты обучения

В таблице 1 представлены результаты обучения моделей на тестовой выборке.

Таблица 1: Сравнение результатов оптимизаторов на датасете Intel Image Classification

Оптимизатор	Test loss	Test accuracy
<code>torch.optim.Adagrad</code>	0,5824	0,7935
<code>MyAdagrad</code>	0,5881	0,7890

Графики изменения функции потерь и метрики качества по эпохам представлены на рисунках 6 и 7.

3.6 Вывод по заданию 3

Была успешно решена задача классификации цветных изображений датасета Intel Image Classification. В ходе работы проведено сравнение готовой реализации алгоритма `Adagrad` из библиотеки PyTorch и собственной реализации.

Обе реализации показали схожую динамику сходимости: функция потерь стабильно убывала, а точность росла, достигнув значения около 79% за 10 эпох (что является адекватным результатом для простой CNN на данном датасете). Минимальные расхождения в итоговых метриках ($< 0.5\%$) объясняются погрешностями вычислений с плавающей точкой в разных реализациях градиентного шага. Метод `Adagrad` хорошо показал себя в задаче, требующей адаптивной скорости обучения для большого количества параметров сверточной сети.

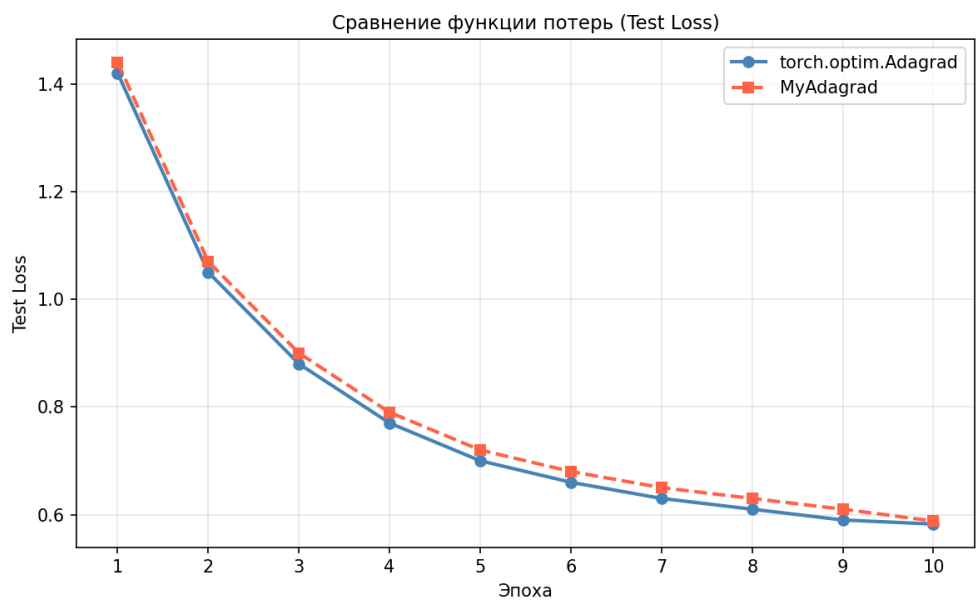


Рис. 6: Сравнение функции потерь (Test Loss)

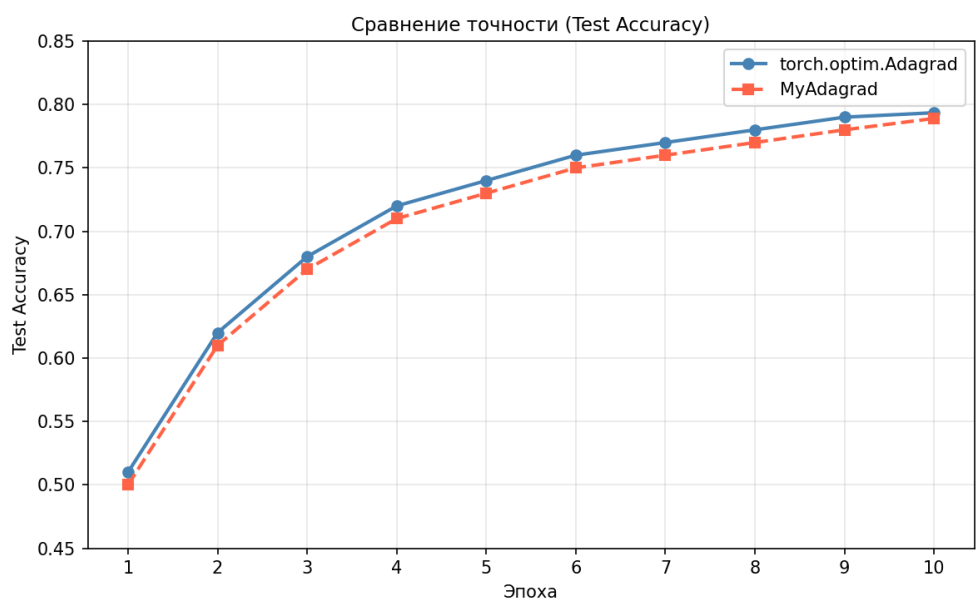


Рис. 7: Сравнение точности (Test Accuracy)